2022 Latin American Robotics Symposium (LARS), 2022 Brazilian Symposium on Robotics (SBR), and 2022 Workshop on Robotics in Education (WRE) | 978-1-6654-6280-8/22/$31.00 ©2022 IEEE | DOI: 10.1109/LARS/SBR/WRE56824.2022.9995748

Collaborative UGV/UAV Path Planning for Inventory Management in Warehouses

Caio C. G. Ribeiro
Department of Computer Science
Universidade Federal de Minas Gerais
Belo Horizonte, Brazil
caioconti@ufmg.br

Leonardo H. M. C. dos Santos
Department of Computer Science
Universidade Federal de Minas Gerais
Belo Horizonte, Brazil
leohmcs@ufmg.br

Douglas G. Macharet
Department of Computer Science
Universidade Federal de Minas Gerais
Belo Horizonte, Brazil
doug@dcc.ufmg.br

***Abstract*—The use of autonomous mobile robots in different applications has increased significantly over the past years, which can be executed individually or collaboratively, considering the use of Multi-Robot Systems (MRS), that presents several advantages but also raises many challenges. Specifically, when considering heterogeneous robots, it is paramount to develop techniques that respect individual restrictions and benefit the use of their different capabilities. In this paper, we consider a ground/aerial collaboration for inventory inspection in warehouse environments. The goal is to determine a set of optimized combined paths that maximize the number of shelves inspected. The problem is formulated as a dual Orienteering Problem (OP), where the budget of both vehicles is considered, and the rewards, which are related to the shelves that can be visited, are estimated by a greedy strategy. Results in simulated scenarios considering different aspects provide a thorough evaluation and validation of the methodology.**

***Index Terms*—Orienteering Problem, Inventory, UAV, UGV**

I. Introduction

Path planning is a crucial step in mobile robot automation, and with the increase of interest in the cooperation between heterogeneous robots, new techniques have arisen. The goal of cooperative path planning is to obtain a path for all vehicles in the system, which often depend on each other. Heterogeneous systems can combine the strengths and weaknesses of different types of robots.

One standard system combines Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs). UAVs are usually a good option for several tasks due to their increased mobility and privileged top perception of the environment; however, their limited flight time can be a huge drawback. On the other hand, UGVs are constrained to the ground but have more carrying capacity and battery life and can spend more time in action without recharge. The cooperation between UAVs and UGVs can create a system that takes advantage of both strengths, the mobility from UAVs and extended battery life from UGVs. This heterogeneous system can be applied to a wide variety of tasks, such as sensor planning in agriculture [1], disaster relief [2], environmental monitoring [3], package delivery [4], [5], among others.

Autonomous mobile robots are a reality in many e-commerce warehouses to maximize efficiency. Robots can be seen moving shelves [6], transporting items [7], and inspecting inventory [8], [9]. We consider the problem of warehouse shelves inspection using a cooperative system between an UGV and UAV. The UGV carries one UAV through the warehouse stopping at selected locations so the UAV can fly and inspect shelves in the area of that stop. The UAV could do the entire mission by itself; however, considering current technology, the battery is very limited, and it would not be able to visit as many shelves as it could using the UGV as transport. In this context, a cooperative approach can take advantage of the strengths of both vehicles. For this, it is then necessary to decide the locations where the UGV will stop and which shelves can be visited by the UAV at each stop and design paths for each vehicle (Figure 1).

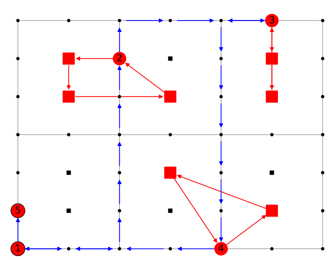


Fig. 1. Red circles are nodes where the UGV stops, numbered by order of visit. Small black dots are aisle nodes where the robot is free to move and selected not to stop. Red squares are shelves marked to visit, while the small black squares are other shelves in the warehouse. Blue arrows are the path made by the UGV and red arrows the paths made by the UAV.

We propose a combined routing approach which considers the budgets of both vehicles and aims to maximize the number of shelves visited. Our contributions are the formulation of the cooperative path planning as an Orienteering Problem (OP) and how we approach the path dependency between the vehicles. In our case, the reward is related to the total shelves visited, and the budget is a limited distance each vehicle can travel. In this scenario, two OP-related problems appear, one for the UAV inspections in each flight and the other for the UGV stops, where one depends on the other. Jointly, our algorithm indicates where the UGV should stop and which

This work was developed with the support of CAPES/Brazil - Finance Code 001, CNPq/Brazil, and FAPEMIG/Brazil.

978-1-6654-6280-8/22/$31.00 ©2022 IEEE

shelves the UAV should visit at every stop. We propose the use of a greedy strategy to solve the UAV-OP, determining which shelves can be visited and assigning the estimated rewards for every stop node; for the UGV-OP, we employ a Genetic Algorithm which determines in which nodes the UGV should stop in order to maximize the number of visited shelves.

II. Related Work

The problem of planning paths is a necessary task for robot automation, and the design of algorithms focusing on heterogeneous systems is one of the main challenges [4] for collaborative robots. Over the past years, several formulations to path planning problems in robotics have been proposed, resulting in strategies that vary depending on the objective.

In that sense, variants of the well-studied Travelling Salesmen Problem (TSP) have been investigated with the objective of minimizing trajectory time or length while visiting all targets of interest [3]–[5], [10]–[12]. In real-world scenarios, however, energy consumption is a very common restriction, as energy is a limited resource in mobile platforms. Therefore, the robots must make the most out of it. In that manner, we tackle this problem using a variant of the TSP, namely OP, which can be seen as a combination of the Knapsack Problem (KP) and TSP [13], with the objective of maximizing reward respecting a budget instead of simply minimizing a single parameter, such as path length or time.

This approach appears to be less frequent in the field of heterogeneous cooperative robots. Works such as [3] and [5] consider a system composed of a carrier ground vehicle and carried aerial vehicles. Although they also consider the budget of limited energy from UAVs, the proposed solutions differ from ours in the sense that they are not trying to maximize a certain reward but minimize pre-defined parameters. Those papers also consider the UGV as having unlimited energy resources and being a recharge station for the UAV. We consider the general case where both the UAV and UGV have a limited budget, and a situation can be created to simulate the UAV recharging.

In [4] and [14], the solutions are based on a Generalized Travelling Salesmen Problem (GTSP) [15] formulation, which also differ from our approach. Mathew *et al.* [4] proposes a solution to the problem of truck-drone package delivery, where the drone is transported by the truck, and both vehicles can deliver. Still, it reduces to a TSP for its final solution with the objective of minimizing the mission time completion. In [4], it is also assumed that the truck has energy for the entire mission, and the aerial vehicle can be recharged by landing on the truck. In [14] is considered the problem where the UAV has to visit all desired sets in the least amount of time using one or more UGVs only as recharging stations and not for transport.

In a more similar manner to our approach, [1] solves the problem of obtaining aerial agriculture measurements using an UGV/UAV coupled system with an OP-based solution to the maximizing the collected rewards problem. Their work solves the OP considering that the UAV has limited energy and the UGV can transport the UAV without the UAV recharging. Nevertheless, they consider that the UGV has an unlimited budget and can travel freely in the continuous space, while in our work, the UGV is limited by its budget, and its movement is limited by a set of fixed nodes that discretizes the warehouse ambient.

Several authors have also studied the problem of surveying shelves in warehouses. In [8] is presented a low-cost sensing system for UAVs autonomous navigation for the warehouse's inventory application. The work in [9] studies the extraction of barcodes using drones for inventory management. Furthermore, multi-robot collaborative use in warehouses was studied by [7] to collect and deliver items. We want to use two collaborative heterogeneous robots to inspect shelves in the warehouse.

We model the warehouse environment as sets of fixed nodes and build one OP for every flight made by the UAV and one OP for choosing the stops for the UGV. The OP is known to be an NP-hard problem implying that finding optimal solutions is time-consuming [16] and can be inviable. Hence, we chose to solve the UAV-OP by a Greedy strategy, taking into consideration that we do not want the UAV to search for long paths. The UGV-OP is solved by a Genetic Algorithm, a known approach for path planning in robotics. The study in [17] proposes an implementation of GA for path planning of cooperative mobile robots. The work in [18] improves the GA crossover operator for designing autonomous robot paths. Kobeaga *et al.* [19] describes an efficient evolutionary algorithm to solve the OP.

III. Problem Formulation

Given $\mathcal{E} \in \mathbb{R}^2$ a grid warehouse-like environment modeled as a graph $G = (V, A)$, where V is set of vertices and $A = \{(i,j) \mid i,j \in V\}$ the set of arcs. The set of vertices is obtained as the combination of two subsets: $P \subset V$, which represents a *roadmap* to the UGV; and $E \subset V$, a *roadmap* to the UAV. More specifically, P defines a set of possible *stopping locations*, *i.e.*, positions where the UGV can release/recover the UAV, and E represents a collection of *shelves*. Furthermore, we also assume an input $E_s \subseteq E$, a subset of shelves selected to be inspected.

The original graph G can be split into two separately graphs, being G_g to the UGV and G_a to the UAV. Both graphs have the same subset of vertices V but differ from the set of arcs A. For G_g, the arcs are created so that every vertex has an arc connecting them with the nearest node in the right, left, up, and down directions, except if the vertex represents a shelf, in that case, the arc is not inserted (the UGV cannot travel through the shelves). In real-world scenarios, the UAV might be able to navigate only in front of the shelves. However, our model considers the UAV can travel more freely in the continuous space, therefore, the arcs in G_a can connect every node to every other node, respecting a pre-defined range, in our case, 2.4m.

As mentioned, we consider a heterogeneous team, composed of a ground vehicle and an aerial vehicle. Each vehicle has a specific travel budget denoted B_G and B_A, respectively, which restricts the distance of actuation. The UAV also has a local flight budget, which is the maximum budget permitted to be used in a single flight, denoted B_{LF}. The ground vehicle is only allowed to move within the stopping nodes P, while the aerial visits nodes in E_s. Finally, we assume the vehicles move in a mutually exclusive manner, *i.e.*, when one is moving the other one is waiting.

In summary, our problem can be defined as:

Problem 1 (Collaborative UGV/UAV Path Planning)**.** *Given an environment $\mathcal{E}$, a heterogeneous team, and a graph G which defines roadmaps for each one of the vehicles, the objective is to maximize the number of shelves in E_s visited by an UAV carried by an UGV, both with limited budget.*

This problem involves the selection of the best stopping nodes (*i.e.*, a subset of P) and their permutation to determine a path for the ground vehicle, in combination with the computation of visiting paths for the aerial vehicle.

IV. Methodology

We formulate the collaborative path planning problem as two separate OPs, the UAV-OP and UGV-OP. In both OPs, the budget is the maximum length the vehicle can travel independently. In our formulation, we usually use a small B_{LF} because we consider that it is not desirable for the drone to travel far away from the surveyed local shelves. The reward assignment for each one is described in A. Besides the budget, the path is also limited by the maximum number of stops S_{max} that the UGV can make, which must be defined before the execution begins. In this work, whenever two non-neighbor nodes need to be connected to form a path, the Dijkstra's algorithm is used to return the shortest path between them. This is mainly used to connect stop nodes calculated for the UGV-OP, building the complete path P. Lastly, it is important to notice that we are not considering the energy being used to land and take off at every stop.

A. Reward assignment/definition

In the OP it is necessary to assign rewards for each node. For the UAV-OP, every shelf has reward of 1 and all others nodes of 0. For the UGV-OP we want to visit most shelves as possible, so it is proposed the reward of a node to be how many shelves can be visited by the UAV if the UGV stops at that node, *e.g.*, if UGV stopped on node 1 and UAV visited 2 shelves, reward from node 1 is 2.

To assign rewards for stop nodes in the UGV-OP, we use Algorithm 1 for every non-shelf node, returning an array of nodes that compose the path E for the UAV for every stop. The function called "Dijkstra" in (Alg. 1) returns the shortest path from the first input to the second input. With every node having their path assigned, to calculate the reward of the node, simply count for every node how many shelves, without repetition, are being visited in that path.

However, the UAV paths assigned to different stop points of P can contain repeated shelves, which is not desired. To

Algorithm 1 Greedy Path Assignment

stop $\leftarrow False$
$S_N \leftarrow$ current_stop {Start Node, current stop node}
$C_N \leftarrow S_N$ {Current Node}
$E \leftarrow \{C_N\}$ {Path Array}
$V_n \leftarrow \{\}$ {Array of visited nodes}
while not stop **do**
    Append C_N to V_n
    $N_e \leftarrow$ Neighbors of C_N
    $N_r \leftarrow N_e$ elements with most reward **not in** V_n
    $C_N \leftarrow$ Closest neighbor of C_N **in** N_r
    if C_N reward $= 0$ **then**
        stop $\leftarrow True$
    else
        $E_{return} = \texttt{Dijkstra}(C_N, S_N)$
        $E_{aux} \leftarrow$ Concatenate $\{C_N\}$ to E
        $B_U =$ Budget used in $E_{aux} + E_{return}$
        if $B_U > B_{LF}$ **then**
            stop $\leftarrow True$
        else
            Append C_N to E
        end if
    end if
end while
return Append $\texttt{Dijkstra}(E(last), S_N)$ to E

correct that, after choosing a sequence of stops that compose P, every shelf visited in every stop is corrected from first to last. If in the current stop a shelf was already visited in a previous stop, that shelf is excluded from the UAV path, and the shelves before and after the removed shelf are connected using Dijkstra's algorithm. In this step, we are adjusting the E first calculated by the greedy algorithm. This procedure of exclusion is not applied to the start and end nodes.

B. Evolutionary algorithm

To solve the UGV-OP, we use a Genetic Algorithm. GAs are inspired by the biological concept of evolution, where the best individuals of a population survive and generate new children to compose the next generations. The method consists of first initializing a random population, then evaluating the individuals by a fitness function. Next, parents are selected to breed and generate new children. Those children can mutate, and finally, a new generation is created using the children and the older population.

C. Encoding of chromosomes

In an environment represented by a graph with N nodes, a possible solution to the problem is represented by one individual called chromosome, denoted by C_h. The chromosome must inform in which nodes the UGV stops and which shelves are visited by the UAV in every stop. In that sense, the chromosome was defined as an array of tuples, each tuple being a gene $g(i)$ that represents a stop to be made by the UGV. Every gene contains the following information:

its unique index i; its (X, Y) coordinates in the 2D graph environment; its reward R; if it is a shelf or a corridor S_c; if it is a node stop forming a path P_v; the order of the stop if it is a stop node O; and the coordinates of the shelves to be visited by the UAV, encoded as the path E, if the UGV stops at that node. In summary,

$$C_h = (g(1), g(2), ..., g(N))$$
$$g(i) = \langle i, (X, Y), R, S_c, P_v, O, E \rangle$$

D. Initialization

In the initialization step, S_{max} non-shelf random nodes are selected to be node stops, and their order is defined by the order of appearance in the selection. This procedure is done until all N_p desired individuals are created to compose the population. Because of the random nature of this procedure, the individuals generated might exceed the budget of both vehicles, hence, being invalid solutions. Furthermore, the path E of every stop node needs to be adjusted, so they do not contain repeated shelves. Therefore, after being created, every individual is corrected by the correction function described in Section IV-G.

E. Fitness Evaluation

The fitness is simply the sum of rewards of all shelves visited in all stop nodes of the solution. If a shelf is visited more than one time, its reward is added only once, *i.e.*, repeated visits to the same shelf don't increase the fitness. In this work, every shelf has the same reward $R = 1$, therefore, the fitness will be numerically equal to the number of visited shelves.

F. Parents Selection and Offspring Generation

The selection operator chooses parents from the population to generate children in the crossover step. The tournament strategy is adopted in this work. This method randomly selects N individuals from the population, and the two with greater fitness are selected to mate.

Crossover is used to mix up two selected parents of the population and generate new individuals. In this work, children are generated by cut-and-fill. Firstly, for every pair of parents, the algorithm randomly chooses if crossover will be performed or not. If it will, a random gene to cut is selected, dividing both parents into two pieces. The first part of the first parent is combined with the second part of the second parent to create the first child. Similarly, the first part of the second parent is combined with the second part of the first parent to create the second child. If the crossover is not performed, both children are returned exactly as their parents.

The mutation is an operator applied to one or more genes in the chromosome used to add new random characteristics to a child. In this work, every child has a chance to mutate and each of its genes that represents a stop node can mutate. The mutation, in our paper, is done by changing the gene to be a non-stop node, effectively taking it off from the UGV stop nodes list. Then its selected randomly a new gene from the whole chromosome to be the new stop node with the same order O than the node before, effectively replacing it.

Finally, all individuals are substituted by the children generated, except for the two individuals with the greatest fitness value of the older generation (elitism).

G. Correction

During the offspring generation, a new individual created can be invalid. To correct that, repeated nodes are excluded and the remaining have their O updated. After that, the set of shelves visited in each stop is corrected following the rule that if the shelf has already been visited at some node before, it is removed from the current node. Then, the corrected individual must be within the UAV and UGV budgets. To guarantee this, if the UGV budget is exceeded, the last stop that isn't the end node is excluded, and the path is remade using Dijkstra. This process repeats until the budget is respected. The same procedure is done in the matter of UAV budget.

V. Experiments

The simulation framework was implemented with Python 3.7. Table I summarizes the experimental parameters used for our GA algorithm in all experiments. The parameters were initially selected based on values commonly used in the literature and adjusted empirically.

TABLE I
GENETIC ALGORITHM PARAMETERS.

Parameter	Value
Population size	200
Number of generations	250
Crossover probability	0.8
Crossover operator	cut-and-fill
Mutation probability (individual)	0.15
Mutation probability (gene)	0.05

A. Illustrative example

Initially, we present an illustrative example for better visualization of the evaluated scenarios and understanding of the methodology and results. Table II shows the parameters.

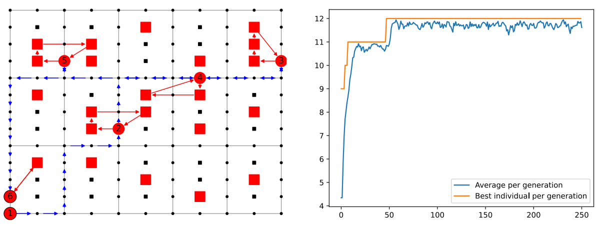
TABLE II
ILLUSTRATIVE EXAMPLE PARAMETERS.

Parameter	Value
Local flight budget	6
UGV budget (used/max)	39.0/40
UAV budget (used/max)	25.77/35
Maximum number of stops	8

For simplicity, in our work, the start point is always the (0,0) node and the final destination the (0,1) node. As they are both considered to be stops, when referring to the S_{max} maximum stops, we mean the random stops calculated by the algorithm plus 2 stops, the start and end nodes.

Figure 2(a) illustrates the path for both UGV and UAV. Red dots are the nodes where the UGV will stop, numbered by order of visit. Black dots are aisle nodes where the robots are free to move and will not stop. Red squares are shelves that we want to visit and black squares are shelves we don't want to visit. The blue arrows trace the path made by the UGV, and red arrows trace the path made by the UAV at every stop. First and last stops are the red dots circled in black.

Regarding the overall performance, Figure 2(b) shows the fitness evolution over the generations, considering the average and best individual's values. As can be seen, the convergence to the final solution generally occurs in less than the max number of generations.



(a) (b)

Fig. 2. Illustrative example. (a) Resulting paths for the UGV (blue) and UAV (red). (b) Fitness evolution over generations.

In this example, the solution was limited by the UGV budget. A shown in Tab. II, the UGV used 39 out of its total budget, which was 40, while the UAV still had budget left, as only 25.77 out of 35 was used. This example demonstrates one of the advantages of our methodology, as it also takes the UGV budget into account when calculating the solution. It is also important to notice that even with the input of maximum of 8 stops, only 6 were made. That happens because stop points are cut out when budgets extrapolate. One point to have in mind is that the whole path made by the UGV may not be an optimal path. That is because as long as the UGV has budget left, it doesn't try to minimize the length travelled, instead, it tries to maximize the reward collected.

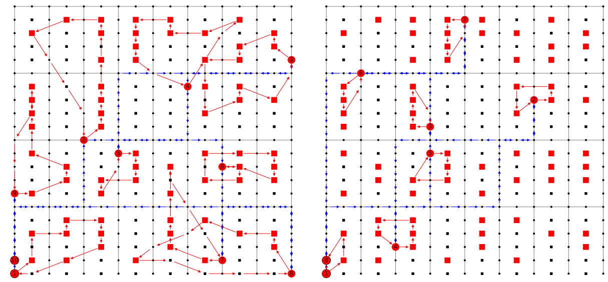
B. *Fixed aerial budget vs. recharging*

In order to simulate a situation where the UAV recharges when landing on the UGV, we consider an unlimited global budget for the UAV, and the local flight budget B_{LF} equal to what would be a reasonable value for its global budget B_a. That way, the local flight budget acts like total drone budget per flight. Parameters for this scenario are shown in Table III.

TABLE III
RECHARGING EXAMPLE PARAMETERS.

Parameter	Value
Local flight budget	50
UAV budget (used/max)	161.27/1000
UGV budget (used/max)	105/150
Maximum number of stops	10

As expected, as long as the UGV budget permits, and enough stops are provided, the system is able to visit all 67 shelves as it did (Figure 3(a)). We can compare the case in the same ambient without recharging. For that, we use 50 as total budget and 8 as local. Without recharging, the system visits less shelves, 26 of 67 (Figure 3(b)).



(a) Recharging. (b) Not recharging.

Fig. 3. Recharging vs not recharging example.

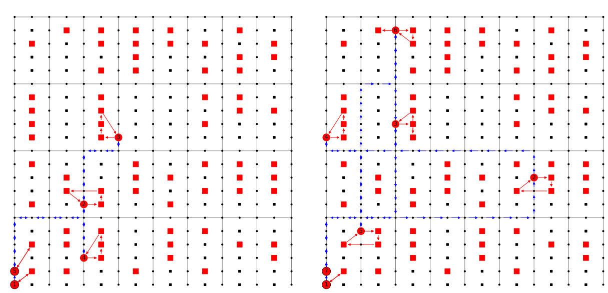
C. *Varying terrestrial budgets*

In order to better illustrate the impact of the UGV budget, we show two examples to compare low and high budgets. The parameters used are shown in Table IV.

TABLE IV
UGV BUDGET COMPARISON PARAMETERS.

Parameter	Value
Local flight budget	6
UAV budget	30
Maximum number of stops	8

In the low budget case, the total B_g was set to 40. In this case, the total B_g used was 39.00 and total B_a used was 23.19, meaning that the number of shelves visited (11 out of 67) was limited by B_g. In the high B_g case, a budget of 600 was set. In this scenario, the use of B_g was 87.0 while B_a was 29.72 out of 30, consequently, visiting as many shelves as it was permitted by its UAV budget (16 out of 67).



(a) Low UGV budget of 40. (b) High UGV budget of 600.

Fig. 4. Low and high UGV budget comparison.

In both cases were made less stops than the maximum, because when the budget is exceeded a stop is cut out.

D. *Quantitative analysis*

Finally, we made a quantitative analysis to evaluate how the aerial and terrestrial budgets independently affect the number of shelves visited. The results for one batch of experiments are summarized in Table V, which shows the percentage of visited shelves from a total of 67 as the UAV and UGV budgets vary. All the tests were done in the same environment with 12 maximum stops and a local flight budget of 8, as these values correspond reasonably well to common vehicles' capabilities. We compared low, reasonable, and high values for each vehicle's budget.

TABLE V
QUANTITATIVE ANALYSIS OF PERCENTAGE OF SHELVES VISITED AS UAV AND UGV BUDGETS CHANGE.

UAV budget \ UGV budget	40	200	600
10	4.5%	4.5%	4.5%
50	16.4%	38.8%	40.3%
200	22.3%	61.2%	62.7%
1000	19.4%	61.2%	62.7 %

The quantitative analysis shows that the system usually grows in the number of shelves visited as the budgets grow, as expected. In the case of the UAV having a budget of 10, it always visits 4.5% of total shelves, not mattering B_g. Although it is possible to travel the whole map with B_g of 600, the low B_a does not let the system visit more shelves. The case with 40 B_g might seem inconsistent, as the number of shelves does not always grow as B_a grows. It happens because the B_g is the limiting factor, hence, growing B_a does not improve the solution. That way, due to the random nature of GA, it could not find the best global solution in all examples, converging into local best. One last important observation is that when both budgets are big enough, the solution does not grow in the number of shelves visited because the limiting factor is the maximum number of stops. It is noticeable that the maximum of 62.7% visited shelves is the best the GA could reach with the limit of 12 stops and local budget of 8.

VI. CONCLUSION AND FUTURE WORK

The use of autonomous mobile robots in logistics can bring several benefits, such as a reduction in costs and delivery time. In this context, warehouse operations, such as inventory inspection, present interesting challenges.

We formulated the problem as an Orienteering Problem, where the reward on each stopping location is determined by a greedy strategy and considers the number of shelves that could be visited. Next, we use an evolutionary algorithm to select the best locations to stop as well as a path connecting them using Dijkstra's algorithm. Results show that our approach is able to generate efficient paths which take full advantage of the team's capabilities.

In future work, instead of considering only the distance traveled, we intend to take into account the time the UGV spends stopped as part of the budget and reformulate the problem also to minimize the task completion time. In this sense, we also plan to allow the vehicles to move simultaneously, making it possible for the UGV to recover the UAV in a different position. Finally, future research directions also include the extension to consider multiple cooperative teams or teams composed of more than two vehicles, *e.g.*, an UGV carrying multiple UAVs.

REFERENCES

[1] P. Tokekar, J. Vander Hook, D. Mulla, and V. Isler, "Sensor planning for a symbiotic UAV and UGV system for precision agriculture," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2013, pp. 5321–5326.

[2] J. Li, G. Deng, C. Luo, Q. Lin, Q. Yan, and Z. Ming, "A Hybrid Path Planning Method in Unmanned Air/Ground Vehicle (UAV/UGV) Cooperative Systems," *IEEE Transactions on Vehicular Technology*, vol. 65, no. 12, pp. 9585–9596, 2016.

[3] S. Narayan, J. Paulos, S. W. Chen, S. Manjanna, and V. Kumar, "Combined Routing and Scheduling of Heterogeneous Transport and Service Agents," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 1466–1472.

[4] N. Mathew, S. L. Smith, and S. L. Waslander, "Planning Paths for Package Delivery in Heterogeneous Multirobot Teams," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 4, pp. 1298–1308, 2015.

[5] H. Y. Jeong, B. D. Song, and S. Lee, "Truck-drone hybrid delivery routing: Payload-energy dependency and No-Fly zones," *International Journal of Production Economics*, vol. 214, pp. 220–233, 2019.

[6] A. Bolu and Ö. Korçak, "Adaptive Task Planning for Multi-Robot Smart Warehouse," *IEEE Access*, vol. 9, pp. 27 346–27 358, 2021.

[7] N. Pinkam, F. Bonnet, and N. Y. Chong, "Robot collaboration in warehouse," in *2016 16th International Conference on Control, Automation and Systems (ICCAS)*, 2016, pp. 269–272.

[8] W. Kwon, J. H. Park, M. Lee, J. Her, S.-H. Kim, and J.-W. Seo, "Robust Autonomous Navigation of Unmanned Aerial Vehicles (UAVs) for Warehouses' Inventory Application," *IEEE Robotics and Automation Letters*, vol. 5, no. 1, pp. 243–249, 2020.

[9] L. Xu, V. R. Kamat, and C. C. Menassa, "Automatic extraction of 1D barcodes from video scans for drone-assisted inventory management in warehousing applications," *International Journal of Logistics Research and Applications*, vol. 21, no. 3, pp. 243–258, 2018. [Online]. Available: https://doi.org/10.1080/13675567.2017.1393505

[10] C. C. Murray and A. G. Chu, "The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery," *Transportation Research Part C: Emerging Technologies*, vol. 54, pp. 86–109, 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0968090X15000844

[11] J. Li, T. Sun, X. Huang, L. Ma, Q. Lin, J. Chen, and V. C. M. Leung, "A Memetic Path Planning Algorithm for Unmanned Air/Ground Vehicle Cooperative Detection Systems," *IEEE Transactions on Automation Science and Engineering*, pp. 1–14, 2021.

[12] L. Amorosi, J. Puerto, and C. Valverde, "Coordinating drones with mothership vehicles: The mothership and drone routing problem with graphs," *Computers & Operations Research*, vol. 136, p. 105445, 2021.

[13] I.-M. Chao, B. L. Golden, and E. A. Wasil, "A fast and effective heuristic for the orienteering problem," *European Journal of Operational Research*, vol. 88, no. 3, pp. 475–489, 1996.

[14] K. Yu, A. Budhiraja, S. Buebel, and P. Tokekar, "Algorithms and experiments on routing of unmanned aerial vehicles with mobile recharging stations," *Journal of Field Robotics*, vol. 36, 12 2018.

[15] G. Laporte and Y. Nobert, "Generalized Travelling Salesman Problem Through n Sets Of Nodes: An Integer Programming Approach," *INFOR: Information Systems and Operational Research*, vol. 21, no. 1, pp. 61–75, 1983.

[16] A. Gunawan, H. C. Lau, and P. Vansteenwegen, "Orienteering problem: A survey of recent variants, solution approaches and applications," *European Journal of Operational Research*, vol. 255, no. 2, pp. 315–332, 2016.

[17] Z. Yu, L. Jinhai, G. Guochang, Z. Rubo, and Y. Haiyan, "An implementation of evolutionary computation for path planning of cooperative mobile robots," in *Proceedings of the 4th World Congress on Intelligent Control and Automation (Cat. No.02EX527)*, vol. 3, 2002, pp. 1798–1802 vol.3.

[18] S. Benhlima, L. Chaymaa, and A. Bekri, "Genetic Algorithm Based Approach for Autonomous Mobile Robot Path Planning," *Procedia Computer Science*, vol. 127, 03 2018.

[19] G. Kobeaga, M. Merino, and J. Lozano, "An efficient evolutionary algorithm for the orienteering problem," *Computers & Operations Research*, vol. 90, 09 2017.